

59
70

TEJAS SAH
BMC202517

Advanced Programming - CU1201
End Semester Exam - March 10, 2026

Total Marks: 70
Duration: 2 hours

Note: For multiple-select questions, full marks are awarded only when all correct options are selected. Partial marks are granted when some, but not all, correct options are selected, provided no incorrect options are chosen. An incorrect selection results in zero mark

1. (1 mark) Are mutable and immutable objects in Python treated differently when passed as arguments to functions?
A. Yes ☒ B. No ☒
2. (1 mark) Which statements correctly describe the difference between class attributes and instance attributes in Python? (Select all that apply)
 - ☒ A. Class attributes are defined at the class level, while instance attributes are typically defined inside the constructor (e.g., `__init__`).
 - ☒ B. Class attributes are shared among all instances of the class.
 - ☐ C. Class attributes cannot be modified after object instantiation, while instance attributes can.
 - ☒ D. Instance attributes are accessed through individual instances, while class attributes can be accessed through the class or any instance.
3. (1 mark) Select the correct statement that describes decorators
 - ☐ A. Decorators can only be applied to functions, not to classes
 - ☐ B. Decorators are user interface elements to beautify the console output
 - ☒ C. Decorators can be stacked to apply multiple enhancements to any function
 - ☐ D. Decorators must modify the functionality of the function it decorates
4. (1 mark) Let $G = (V, E)$ be a simple undirected graph. The weights of the edges are positive and distinct. The statement *shortest path between any two vertices of G is always unique* is
A. True ☒ B. False ☒
5. (1 mark) Consider a directed graph G with 600 edges with the least number of vertices possible. What will be the number of vertices in graph G ? 25
6. (2 marks) A non-negative integer is passed as an input parameter to the recursive function `factorial`. Given an input of 10, what is the maximum depth of the call stack? 10 (if returns at 1, 11 if returns at 0)
7. (4 marks) An undirected graph G has 17 vertices. The sum of the degrees of all the vertices in G is D . The number of vertices of even degree in G is K . Which of these values are possible for D and K ?
A. $D = 41, K = 9$ B. $D = 41, K = 10$ C. $D = 42, K = 9$ D. $D = 42, K = 10$
correct.

B. $D = 41, K = 10$ C. $D = 42, K = 9$	Not $D = 41, K = 9$ as sum of K degrees is even, so $D - \sum(\text{deg}) = \text{odd}$ $\therefore D - K$ are even only $(17 - 9) = 8$ $\rightarrow$ $D = 42, K = 10$ is correct.
--	---
8. (2 marks) We can use Breadth First Search (BFS) instead of Dijkstra's algorithm to find out the shortest path from the given source node to every other node only C.
 - ☐ A. When all the edge weights are strictly greater than 1.
 - ☐ B. When all the edge weights are distinct.
 - ☒ C. When all the edge weights are equal
 - ☐ D. When graph has cycles

9. (4 marks) The following cache decorator intends to memoize results of a single-argument function. When the program given below is run, the program gives a type error.

`TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'`. Fix the code.

```
def cache(func):
    memory = {}
    def wrapper(n):
        if n not in memory:
            memory[n] = func(n)
            return memory[n]
    return wrapper

@cache
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(10))
```

Line 6: `return memory[n]` should be indented less.

↳ It should not be inside the if condition, ~~out~~ but outside it, in the wrapper.

✓

10. (3 marks) A software developer was tasked with creating a Python function named `twice`. This function should accept a list of integers as input and return a set containing all integers that appear exactly twice in the list. The developer's initial implementation is provided below for reference. Correct any bugs in the code and post the response in the box given below. Alternatively, you may write **The code has no bug** in the box provided below.

```
def twice(num_list):
    num_counts = {}
    for num in num_list:
        num_counts[num] += 1
    return {num for num, count in num_counts.items() if count == 2}

if __name__ == "__main__":
    data = [1, 3, 1, 3, 3, 2, 4, 2, 2]
    result = twice(data)
    print(result)
```

Before line 4, add:

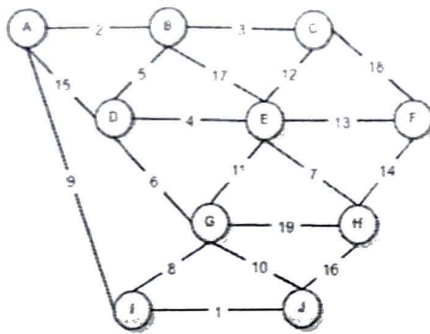
`if num not in num_counts:`
`num_counts[num] = 0`

to then return

✓

3

11. Consider the graph given below to answer the next two questions



(a) (2 marks) Write (in order) the list of vertices that would be visited by running a breadth-first search (BFS) starting from G.

(b) (2 marks) What is the fifth node visited if DFS algorithm is used to find the shortest path from G?

a) [G, D, E, H, I, J, A, B, C, F]

b) G → D → E → C → B

(For a), in BFS, BFS did not enforce weights at all) → Build to 5th visited node

→ But if it is, then order:

[G, D, I, J, E, H, B, A, C, F]

12. (5 marks) Consider the following code and answer the questions given below:

```

1 def is_valid_stack(operations):
2     stack = []
3     for operation in operations:
4         if #Fill in the missing code segment in the box given below
5         ...
6         elif operation == "pop":
7             if not stack:
8                 return False
9             print(stack.pop())
10    return not stack
11
12 if __name__ == "__main__":
13     operations = ["push 1", "push 2", "pop", "push 1", \
14                 "push 2", "pop", "pop", "pop", "push 2", "pop"]
15     if is_valid_stack(operations):
16         print("Valid stack operations")
17     else:
18         print("Invalid stack operations")
19

```

Write the missing code. Use standard operations of stack

Ans: if "push" in operation:
stack.append(int(operation.split(" ")[1]))

13. (2 marks) The provided program is intended to implement a basic stack data structure with push and pop operations. Select all the correct implementation.

☒ A. class Stack:

```

    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        return self.items.pop()
```

B. class Stack:

```

    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        return self.items.pop(0)
```

This is also correct

☐ C. class Stack:

```

    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.insert(0,x)
    def pop(self):
        return self.items.pop(0)
```

D. class Stack:

```

    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.insert(0,x)
    def pop(self):
        return self.items.pop()
```

14. (4 marks) The provided function, count_nodes, employs a recursive depth-first search (DFS) algorithm to count the nodes within a binary tree. However, upon execution, it encounters an UnboundLocalError indicating that the local variable count is not associated with a value. To rectify this issue, the program must be modified so that the function count_nodes accurately outputs the correct count of nodes within the binary tree.

```

1 class TreeNode:
2     def __init__(self, val=0, left=None,
3         right=None):
4         self.val = val
5         self.left = left
6         self.right = right
7
8 def count_nodes(root):
9     count = 0
10    def dfs(node):
11        if not node:
12            return
13        count += 1
14        dfs(node.left)
15        dfs(node.right)
16    dfs(root)
17    return count
18
19 #Sample usage
20 root = TreeNode(1)
21 node2 = TreeNode(2)
22 node3 = TreeNode(3)
23 node4 = TreeNode(4)
24
25 root.left = node2
26 root.right = node3
27 node2.left = node4
28
29 print(count_nodes(root)) #4
```

*→ Insert between line 9 and 10,
nonlocal count*

→ See

*def count_nodes(root):
 count = 0
 def dfs(node):
 nonlocal count
 if not node:
 return*

4

15. (4 marks) The remove method should unlink the node containing value. Currently it mistakenly links the previous node to the found node instead of skipping it, corrupting the list. Fix the code.

```

class Node:
    def __init__(self, v):
        self.value = v
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, v):
        node = Node(v)
        if not self.head:
            self.head = node
        return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = node

    def remove(self, value):
        cur = self.head
        prev = None
        while cur and cur.value != value:
            prev = cur
            cur = cur.next
        if not cur:
            return # value not found
        if prev:
            prev.next = cur
        else:
            self.head = cur
    
```

Line 10:
prev.next = cur.next
Line 12:
self.head = cur.next

16. A library stores book records as tuples: (title, author, year, copies_available). Given a list books, write:

- (2 marks) a list comprehension that returns titles of books published after 2010
- (2 marks) a dictionary comprehension mapping author:total copies available (sum across their books)
- (2 marks) a set comprehension of all publication years present.

Write the comprehensions below.

```

books = [
    ("Deep Learning", "Goodfellow", 2016, 3),
    ("Python Tricks", "Brett Slatkin", 2017, 5),
    ("Intro to Algorithms", "Cormen", 2009, 2),
    ("Fluent Python", "Luciano Ramalho", 2015, 1),
    ("Some Old Book", "Anon", 1995, 4),
]
    
```

- (a) titles after 2010. Complete the statement using comprehension

titles_after_2010 = [titles for (titles, author, year, copies) in books if year >= 2010]

- (b) author: total copies - Complete the statement using comprehension

copies_by_author = {name: sum([copies for (title, name, year, copies) in books if name == name]) for name in set([name for (title, name, year, copies) in books])}

- (c) set of publication years - Complete the statement using comprehension

years = {year for (title, author, year, copies) in books}

b) copies-by-author = {name: sum([copies for (title, name, year, copies) in books if name == name]) for name in set([name for (title, name, year, copies) in books])}

b) copies-by-author = {name: sum([copies for (title, name, year, copies) in books if name == name]) for name in set([name for (title, name, year, copies) in books])}

Multi-occurrence of author.

17. (4 marks) The provided comprehension code endeavors to construct a dictionary that establishes a mapping between the variable `student` and the variable `average score`. This mapping is derived from a list of tuples, each containing a `student` and a corresponding `score`. There are two distinct and correct implementations of this code. Provide the correct order of the output for both attempts.

```
records = [("ram", 80), ("raj", 90), ("ram", 70), ("raj", 85), ("radhika", 100)]
```

two attempts:

```
avg_by_student_1 = {s: sum([v for (x,v) in records if x==s]) / len([v for (x,v) in records if x==s]) for (s,_) in records}
```

```
avg_by_student_2 = {s: sum(v for (x,v) in records if x==s) / len([v for (x,v) in records if x==s]) for s in {x for (x,_) in records}}
```

Handwritten analysis of the two dictionary comprehensions:

- avg-by-student-1** order: `from: avg`, `raj: avg`, `ram: avg`, `raj: avg`, `radhika: avg`. It is noted that this "overrides, but some values" (circled 4).
- avg-by-student-2** order: `from: avg`, `raj: avg`, `radhika: avg`. This is marked with a checkmark and a circled 5.

18. (3 marks) Write a function `safe_divide_list` with appropriate parameters that divides each number in `nums` by `denom` and returns a list of results. It should catch `ZeroDivisionError` and return the string `DIV_BY_ZERO`. Fill in the function body in the box given below.

```
# Example:
safe_divide_list([10,20,30], 10) # [1.0, 2.0, 3.0]
safe_divide_list([10,20], 0) # "DIV_BY_ZERO"
```

Handwritten function definition:

```
def safe_divide_list(list, denom):
    try:
        return [x/denom for x in list]
    except ZeroDivisionError:
        return "DIV_BY_ZERO"
```

Marked with a checkmark and circled 3.

19. (4 marks) Write a decorator `log_calls` that prints the function name and arguments before calling the function, and returns the function result. Fill in the wrapper body in the skeleton below (replace the `pass` line). Provide only the wrapper body (1-4 lines).

```
@log_calls
def add(a, b):
    return a + b

# Example behaviour:
add(2,3) #Should print 'Calling add with args: (2,3) kwargs={} and return 5'
```

Handwritten decorator implementation:

```
def wrapper(*args, **kwargs):
    print(f"calling {func.__name__} with args: {args} kwargs: {kwargs}")
    value = func(*args, **kwargs)
    print(value)
    return value
```

Marked with a checkmark and circled 4.

Additional note: "# being print without a new line, as I don't know the function output before calling it; and question asks to print before calling"

20. (4 marks) The BankAccount class utilizes methods to encapsulate behavior and safely modify internal state. A critical bug in the withdraw method currently bypasses the data protection mechanisms. Malicious users have discovered they can pass negative amounts to the withdraw method, which effectively acts as an unchecked deposit and increases their balance.

Review the class below. Identify and correct the logical bug in the code so that withdrawals are strictly validated.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return amount
        print("Insufficient funds")
        return 0
```

Sample Call:

```
account = BankAccount("Ram", 500)
account.withdraw(-5000) # Malicious
                        withdrawal attempt
Expected Output after the fix:
Insufficient funds
```

→ Change line 11 to
 if $0 \leq \text{amount} \leq \text{self.balance}$:
 Although Amount shd not be 0 also

21. (5 marks) Write a Python dictionary where the keys and values are prime numbers between 2 and 100, utilizing only a dictionary comprehension. Full marks (5) will be awarded if the comprehension is written without the use of a separate prime function; otherwise, only 1 mark will be given.

Can use list comprehension in the code?
 → No

Method 1

```
Primes = {1:2, 2:3}
for i in range(4, 100):
    for p in Primes.values():
        if i % p == 0:
            break
    else:
        # also use for, not if, here
        Primes[Primes[Primes[1]] + 1] = i
```

Method 2 (no loops either)

```
{i: Prime for i, Prime in enumerate(
    [3, 5, 7, 11, 13, 17, 23, ...
    ..., 97])}
```

Method 3: Can use list comprehensions!

```
{i: Prime for i, Prime in enumerate(
    [P for P in range(2, 100) if
    not (P%2==0 or P%3==0 or P%5==0 or P%7==0 or P%11==0 or P%13==0 or P%17==0 or P%19==0 or P%23==0 or P%29==0 or P%31==0 or P%37==0 or P%41==0 or P%43==0 or P%47==0 or P%53==0 or P%59==0 or P%61==0 or P%67==0 or P%71==0 or P%73==0 or P%79==0 or P%83==0 or P%89==0 or P%97==0)])}
```

Method 4:

```
Primes = {i+5: P for i, P in enumerate(range(5, 100))
    if not (P%2==0 or P%3==0 or P%5==0 or P%7==0 or P%11==0 or P%13==0 or P%17==0 or P%19==0 or P%23==0 or P%29==0 or P%31==0 or P%37==0 or P%41==0 or P%43==0 or P%47==0 or P%53==0 or P%59==0 or P%61==0 or P%67==0 or P%71==0 or P%73==0 or P%79==0 or P%83==0 or P%89==0 or P%97==0)}
```

Primes[1]=2
 Primes[2]=3
 Primes[3]=5
 Primes[4]=7

Primes2 = {i+1: P for i, P in enumerate(sorted(Primes.values()))}

Not indexed correctly though:

X (You have to output {p: p} type of dictionary where p is some prime b/w 2 to 100 not with prime in p's)

22. A search engine company is creating a web crawler to understand how websites are connected. This crawler uses an iterative Depth First Search (DFS) approach, which relies on a stack. The stack allows the crawler to pause its current path and explore deeper links first. However, the developer forgot to delete the code and submitted without testing.

- Complete Logic to correctly add valid neighboring websites to the stack for further exploration.

A **neighboring website** is any website that is directly linked from the current page. In other words, if the crawler is visiting Website A and finds a hyperlink pointing to Website B, then Website B is considered a neighboring website of Website A. These neighboring websites are the next nodes in the DFS traversal and are pushed onto the stack for further exploration.

A **crawler**, also known as a web crawler or spider, is a program used by search engines to automatically browse the internet and collect data from websites. It systematically moves from one webpage to another by following hyperlinks, indexing the content it finds to help build an organized search database.

- (a) (3 marks) complete the code in the box given below
(b) (2 marks) What is the output for the test code given below

```

1 def map_web_dfs(graph, start):
2     if start not in graph: return []
3
4     visited = set()
5     stack = [start]
6     result = []
7
8     while stack:
9         node = stack.pop()
10        if node not in visited:
11            visited.add(node)
12            result.append(node)
13
14            neighbors = sorted(graph.get(node, []), reverse=True)
15            for neighbor in neighbors:
16                #write the missing code in the box given below
17
18    return result
    
```

Test case:

```

1 graph = {0: [6, 12], 1: [12], 2: [6], 3: [13], 4: [14]}
2 print(map_web_dfs(graph, 0))
    
```

*for neighbors in neighbors:
if not (neighbors in visited) and not (neighbors in stack):
stack.append(neighbors)*

Test case output: [0, 6, 12]

5

